

Data Augmentation with Generative AI

Text, Image, Audio & Tabular — Additional Lecture 1 of 3

Ezequiel López-Rubio

Tuesday, September 22, 2026

Malmö University, Sweden

DA631E — Artificial Intelligence for Data Science

Where these three lectures sit

You have just finished **Lecture 6: Introduction to ML**. Next comes **Lecture 7: Validation & Evaluation**. In between, three lectures on how **Generative AI** reshapes the *data* side of machine learning (ML):

1. **Data augmentation** across modalities (today)
2. **Synthetic data generation** across modalities
3. **Hands-on workshop**

Guiding idea

A model is only as good as its data. Before we *evaluate* models (Lecture 7) or *train* them in depth (Lectures 8–9), let us learn to *improve the data* they learn from — for text, images, audio, and tables.

What we assume today

We build on

- Data collection & preprocessing (L4–5)
- The ML pipeline / training set idea (L6)
- Basic Python

We do *not* assume

- How generative models work inside (Large Language Models (LLMs): L13; neural nets: L12)
- Formal evaluation (that is L7 — today we only *peek*)

Our stance

We treat generative models as **black-box tools** we *prompt* or *steer*. We keep every experiment **small** so free Google Colab never runs out of resources.

1. The data bottleneck
2. Augmentation across modalities
3. Pitfalls and good practice
4. Practical in Google Colab
5. Wrap-up

The data bottleneck

Data is the fuel of machine learning

- ML recipe (L6): **data** → **model** → **predictions**.
- Quality *and* diversity of labelled data drive performance.
- Good labelled data is often **scarce, expensive, or slow** — in **every** modality.

The bottleneck looks different per modality

- **Text**: few labelled documents; rare intents.
- **Image**: costly annotation; few examples of rare classes.
- **Audio**: expensive to record; few speakers / conditions.
- **Tabular**: class imbalance; privacy-locked records.

Two ways to get more out of your data

Augmentation (today)

Transform existing labelled examples into new, plausible variants.

A photo → a rotated photo.

Synthetic data (next)

Create new examples from a description or schema.

“Invent” 200 photos that never existed.

Same goal (more useful training data), different mechanism.

Augmentation across modalities

Definition and the golden rule

Data augmentation

Enlarging a training set with **label-preserving transformations** of existing examples, to improve generalisation and robustness.

Golden rule: preserve the label.

- A rotated cat photo is still a cat.
- A pitch-shifted “yes” is still “yes”.
- A rephrased positive review is still positive.

If a transformation can flip the label, it is **not** valid augmentation.

Classic transformations you already have (no LLM needed)

Modality	Cheap, label-preserving transforms
Image	flip, rotate, crop, colour jitter, blur, noise
Audio	time-stretch, pitch-shift, add noise, time-shift
Tabular	feature jitter, Synthetic Minority Oversampling Technique (SMOTE) interpolation for the minority class
Text	synonym swap, back-translation, small edits

These are **fast and free**. Start here — always.

The cost–power spectrum in the GenAI era

Key lesson

Augmentation ranges from **cheap transforms** to **costly generation**. Pick the *lightest* tool that still adds useful, label-preserving variety.

cheap / robust $\iff$ powerful / expensive

Digital Signal Processing (DSP) & transforms $\rightarrow$

SMOTE $\rightarrow$ LLM paraphrase $\rightarrow$ diffusion / Text To Speech (TTS)

Today: LLMs for **text & tabular text**, diffusion (few-step) for **images**; DSP for **audio** (generative audio is heavier — we *generate* audio in Lecture 2).

Generative augmentation, per modality

Text LLM **paraphrase** / style transfer / class-conditional rewriting.

Tabular SMOTE for numeric imbalance; **LLM** to rewrite free-text columns.

Image few-step diffusion **img2img** to vary lighting/angle while keeping the object.

Audio DSP transforms today; generative re-synthesis (TTS) is heavier (Lecture 2).

Control knobs

Prompts (what to keep/change), *sampling temperature* (text), *strength* (img2img).
Higher variety $\Rightarrow$ higher risk of **label drift**.

Fidelity vs. diversity: the core trade-off

Too conservative

- Near-duplicates of the seed.
- Adds little new signal.

Too creative

- Drifts off-topic.
- Flips the label (**label drift**).

Good augmentation lives in the middle: **new form, same label**.

Pitfalls and good practice

What can go wrong (all modalities)

- **Label drift:** an over-strong transform changes the class.
- **Hallucination:** generated content invents facts / artefacts.
- **Distribution shift:** augmented data looks unlike real inputs.
- **Redundancy:** many near-identical variants inflate size, not information.
- **Leakage:** an augmented copy of a *test* item lands in training.

Good practice checklist

1. **Split first, augment second.** Only ever augment the **training** split.
2. Keep *provenance* (original vs. augmented flag).
3. Spot-check samples by hand for label drift.
4. De-duplicate; cap variants per seed.
5. **Manage compute:** load one model at a time; free memory; keep counts tiny.
6. Mind **governance:** licences, privacy, General Data Protection Regulation (GDPR, EU/Sweden).

Bridge to Lecture 7

“Did augmentation help?” is an **evaluation** question — measured properly next lecture. Today we only *peek*.

Practical in Google Colab

Practical: plan and constraints

Goal. Augment a tiny dataset in *each* modality and inspect the result.

Ground rules (free-Colab friendly)

- Everything in **Google Colab**; no local install.
- Enable free GPU: *Runtime* → *Change runtime type* → *T4 GPU*.
- A **handful** of examples per modality — this is a demo, not a benchmark.
- Load models **one modality at a time** and **free memory** in between.

Practical: a memory helper (run once)

```
import gc, torch

def free_memory(*objs):
    """Delete large objects and clear the GPU cache between sections."""
    for o in objs:
        try: del o
        except: pass
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()

print("GPU available:", torch.cuda.is_available())
```

Call `free_memory(model)` after each modality to stay within free-tier limits.

Modality 1 — Text: a small flan-t5 paraphraser

```
!pip -q install transformers sentencepiece scikit-learn imbalanced-learn

# The "text2text-generation" pipeline task was removed in recent
# transformers, so we call flan-t5 through the stable model API.
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

tok = AutoTokenizer.from_pretrained("google/flan-t5-base")
model = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-base",
                                              torch_dtype=torch.float16 if torch.cuda.is_available() else torch.float32
                                              ).to("cuda" if torch.cuda.is_available() else "cpu")

def t2t(prompt, n=1, temperature=1.1, max_new_tokens=60):
    x = tok(prompt, return_tensors="pt", truncation=True).to(model.device)
    out = model.generate(**x, do_sample=True, top_p=0.95, temperature=temperature,
                        max_new_tokens=max_new_tokens, num_return_sequences=n)
    return [tok.decode(o, skip_special_tokens=True).strip() for o in out]
```

One model serves the text *and* tabular sections; we free it once, afterwards.

Text: paraphrase and build the augmented set

```
import pandas as pd

seed = [("The battery lasts all day and charges fast.", "positive"),
        ("Terrible support, I waited two hours on hold.", "negative")]

def paraphrase(text, sentiment, n=2):
    p = (f"Paraphrase this {sentiment} review; keep the same sentiment, "
         f"change the wording. Review: {text}")
    return t2t(p, n=n, temperature=1.1, max_new_tokens=60)

rows = []
for t, y in seed:
    rows.append((t, y, "original"))
    for v in paraphrase(t, y, n=2):
        if v and v.lower() != t.lower():
            rows.append((v, y, "augmented"))
df_text = pd.DataFrame(rows, columns=["text", "label", "source"])
print(df_text["source"].value_counts())
```

source records provenance. We **keep** the model — the tabular section reuses it.

Modality 2 — Tabular: SMOTE for class imbalance

```
import numpy as np, pandas as pd
from imblearn.over_sampling import SMOTE

# tiny imbalanced numeric table: 9 "stay" vs 3 "churn"
X = pd.DataFrame({
    "age":      [25,34,52,41,29,60,38,47,23,31,55,44],
    "tenure":   [ 2,18,40,12, 5,55,20,33, 1, 9,48,15],
    "spend":    [30,55,80,60,25,90,50,70,20,35,85,58]})
y = np.array(["stay"*9 + "churn"*3])

Xr, yr = SMOTE(k_neighbors=2, random_state=0).fit_resample(X, y)
print("before:", np.bincount((y=="churn")), " after:", np.bincount((yr=="churn")))
```

SMOTE *interpolates* new minority rows — the classic, zero-LLM tabular augmenter.

Tabular: LLM augmentation of a free-text column

```
# Many real tables contain TEXT fields; augment those with the LLM.
hr = pd.DataFrame({
    "dept": ["sales", "sales", "eng"],
    "score": [4.6, 3.1, 4.9],
    "note": ["Consistently exceeds quarterly targets.",
            "Struggled to meet deadlines this quarter.",
            "Outstanding technical contributions to the team."],
    "promote": ["yes", "no", "yes"]})

def rephrase(note): # reuse the flan-t5 model from the text section
    return t2t(f"Paraphrase, keep the same meaning: {note}",
               n=1, temperature=1.0, max_new_tokens=40)[0]

hr_aug = hr.copy(); hr_aug["note"] = hr_aug["note"].map(rephrase)
hr_aug["source"] = "augmented"
print(hr_aug[["note", "promote", "source"]])
free_memory(model, tok) # release flan-t5 before the image model
```

Modality 3 — Image: few-step diffusion img2img

```
!pip -q install diffusers accelerate
from diffusers import AutoPipelineForImage2Image
from sklearn.datasets import load_sample_image
from PIL import Image

img = Image.fromarray(load_sample_image("flower.jpg")).resize((512, 512)) # label: "flower"

pipe = AutoPipelineForImage2Image.from_pretrained(
    "stabilityai/sd-turbo",
    torch_dtype=torch.float16 if torch.cuda.is_available() else torch.float32)
pipe = pipe.to("cuda" if torch.cuda.is_available() else "cpu")
```

sd-turbo is a **few-step** model — fast and light enough for a short demo.

Image: generate label-preserving variants

```
variants = []  
for i in range(3):  
    out = pipe("a photo of the same flower, different lighting and angle",  
              image=img, num_inference_steps=4, strength=0.55,  
              guidance_scale=0.0)      # sd-turbo: guidance 0  
    variants.append(out.images[0])  
  
variants[0]      # display the first variant in Colab  
free_memory(pipe)
```

Watch the strength knob

Low strength → near-copies. High strength → the object (label) changes. Too much diversity becomes **label drift**. A classical fallback:

from torchvision import transforms → flip / rotate / colour-jitter (CPU, free).

Modality 4 — Audio: DSP augmentation (light)

```
!pip -q install audiomentations librosa soundfile
import librosa
from audiomentations import Compose, AddGaussianNoise, TimeStretch, PitchShift
from IPython.display import Audio

y, sr = librosa.load(librosa.example("trumpet"))    # small sample clip

augment = Compose([
    AddGaussianNoise(min_amplitude=0.001, max_amplitude=0.01, p=1.0),
    TimeStretch(min_rate=0.9, max_rate=1.1, p=1.0),
    PitchShift(min_semitones=-2, max_semitones=2, p=1.0)])

y_aug = augment(samples=y, sample_rate=sr)
Audio(y_aug, rate=sr)    # listen: is it still recognisably the same sound?
```

Label-preserving means modest shifts. ± 2 semitones keeps a spoken word intelligible; ± 12 would not.

Generative audio comes in Lecture 2.

A first “peek”: does text augmentation help?

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline

test = [("Fast, reliable, worth every penny.", "positive"),
        ("Awful quality, it broke immediately.", "negative")]
Xte, yte = zip(*test)
def acc(train):
    clf = make_pipeline(TfidfVectorizer(), LogisticRegression(max_iter=1000))
    clf.fit(train["text"], train["label"]); return clf.score(Xte, yte)

print("original only :", acc(df_text[df_text.source == "original"]))
print("original + aug :", acc(df_text))
```

Caveat: two test points prove nothing. *Proper* evaluation = Lecture 7.

Practical: what to observe and discuss

- **Text:** any paraphrase that flips sentiment? How does temperature change variety?
- **Tabular:** are SMOTE rows plausible? Did the LLM keep the note's meaning?
- **Image:** at which strength does the flower stop being a flower?
- **Audio:** how far can you pitch-shift before the label breaks?
- Why is testing on two hand-picked items **not** a fair evaluation?

Wrap-up

Key takeaways

1. Augmentation = **transform** existing data with **label-preserving** edits.
2. It applies to **text, image, audio, and tabular** — with different tools.
3. Use the **lightest** method that adds useful variety (transforms → generation).
4. Watch for **label drift, duplicates, leakage**; split *before* you augment.
5. Manage **compute/memory**; whether it helps is an evaluation question (L7).

Additional Lecture 2 — Synthetic Data Generation with Generative AI

From *transforming* data to *creating* it from scratch — text, tabular, images (diffusion) and audio (TTS), plus how to tell if it is any good.

Questions?

Further reading (optional)

- Shorten & Khoshgoftaar (2019), *A Survey on Image Data Augmentation*.
- Feng et al. (2021), *A Survey of Data Augmentation Approaches for NLP*.
- Dai et al. (2023), *AugGPT: Leveraging ChatGPT for Text Data Augmentation*.
- Chawla et al. (2002), *SMOTE*; Park et al. (2019), *SpecAugment* (audio).
- Docs: Hugging Face transformers & diffusers; audiomentations.